



PROJECT BRIEF

Cahier des Charges

Plateforme de gestion de tableaux et collaboration

Version : 1.0 • Date : Février 2026

Sommaire

1. Résumé exécutif

2. Contexte, vision et objectifs

2.1 Contexte

2.2 Vision produit

2.3 Objectifs fonctionnels

2.4 Objectifs techniques

3. Périmètre fonctionnel (actuel + cible)

3.1 Déjà en place

3.2 Cible proche

3.3 Évolutions prévues

4. Architecture technique

4.1 Frontend

4.2 Backend

4.3 Infrastructure

5. Benchmark technologique (choix justifiés)

5.1 Frontend

5.2 Backend

5.3 API

5.4 Base de données

5.5 Infra

6. Environnements

7. CI/CD & Qualité

8. Organisation du projet (Git/GitHub)

9. Plan de livraison (macro)

10. Risques & mitigations

11. Livrables

Document évolutif, mis à jour au fil des itérations.

1. Résumé exécutif

EpiTrello est une application de gestion de projets inspirée de Trello, centrée sur des tableaux kanban, des workspaces et la collaboration. Le produit vise une expérience fluide et moderne, avec une base technique scalable et une organisation de projet rigoureuse (CI/CD, branches par feature, environnements dev/prod).

L'objectif est de livrer une première version solide (auth, tableaux, workspaces, paramètres) tout en préparant les évolutions stratégiques : temps réel, templates de tableaux, automatisations et notifications.

Le projet s'appuie sur une stack web moderne (Next.js, GraphQL, Prisma, PostgreSQL) et une infrastructure reproductible via Docker et Terraform. Les environnements dev/prod sont séparés, et la chaîne CI/CD intègre la couverture de code backend pour sécuriser les évolutions.

L'organisation du travail se fait par features (branches dédiées, issues et PR) afin de maintenir une cadence claire et une traçabilité des changements.

2. Contexte, vision et objectifs

2.1 Contexte

Le besoin est de fournir un outil de pilotage simple et efficace, adapté à un usage académique et à un usage produit réel. Le projet doit également démontrer une capacité à concevoir une architecture complète (front, back, infra) et à l'exploiter via un pipeline CI/CD.

2.2 Vision produit

Offrir une alternative claire et performante à Trello : interface cohérente, navigation rapide, et collaboration au cœur du flux de travail. Le produit doit être agréable dès le premier usage, tout en restant extensible.

2.3 Objectifs fonctionnels

- Gestion intuitive des tableaux, listes et cartes.
- Collaboration par workspaces (membres, rôles, permissions).
- Authentification moderne (email/mdp et OAuth).
- Paramètres utilisateur complets (profil, accessibilité, sécurité).

2.4 Objectifs techniques

- Stack front moderne (Next.js, composants réutilisables, design system).
- API GraphQL robuste et typée.
- Infrastructure reproductible (Docker, Terraform, GCP).
- Qualité logicielle via CI/CD et couverture de code côté backend.

3. Périmètre fonctionnel (actuel + cible)

Déjà en place

- Frontend Next.js (pages landing, login/signup, account settings, boards).
- Sidebar boards avec favoris/récents/workspaces.
- Pages de paramètres utilisateur (profil, activité, paramètres, accessibilité).
- Navbar commune.

Cible proche

- Auth OAuth (GitHub, Discord) + email/mdp.
- Gestion complète des workspaces (membres, rôles, paramètres, danger zone).
- Gestion des boards (création, visibilité, templates).

Évolutions prévues

- Temps réel via WebSocket (cartes, listes, commentaires).
- Templates de boards.
- Notifications et activité.
- Filtrage avancé et recherche.

4. Architecture technique

Frontend

- Next.js 14 + React 18.
- Tailwind CSS + shadcn/ui.
- Apollo Client pour GraphQL.

Backend

- Node.js + Express + Apollo Server (GraphQL).
- Prisma ORM.
- PostgreSQL.

Infrastructure

- Docker Compose (dev/prod).
- Terraform pour l'infra.
- Provider cloud : Google Cloud Platform (GCP).

5. Benchmark technologique (choix justifiés)

Frontend

- **Next.js** vs Vite+React: Next.js apporte SSR/SSG, routing intégré, et un déploiement simple. Vite est plus léger mais nécessite plus d'assemblage.

Backend

- **Apollo Server + Express** vs NestJS: Apollo/Express est plus direct et léger pour GraphQL, NestJS apporte davantage de structure mais peut être plus lourd pour ce besoin.

API

- **GraphQL** vs REST: GraphQL réduit le sur-fetching, permet de composer les vues rapidement et convient bien aux interfaces complexes (boards, cartes, membres).

Base de données

- **PostgreSQL** vs MySQL/MongoDB: PostgreSQL est robuste pour les relations (workspaces, boards, permissions) et bien supporté par Prisma.

Infra

- **GCP + Terraform** vs alternatives (AWS/Azure): choix motivé par la capacité à automatiser l'infra via Terraform et la cohérence avec les environnements de déploiement visés.

6. Environnements

- **Dev** : Docker Compose dev + variables `env.dev.template` .
- **Prod** : Docker Compose prod + variables `env.prod.template` .

7. CI/CD & Qualité

- CI/CD avec pipeline de build/test/deploy.
- Intégration **Codecov** pour couverture backend.
- Déploiements automatisés via Terraform (GCP).

8. Organisation du projet (Git/GitHub)

- Branching par **feature**.
- `main` = branche de production.
- PR obligatoires, review avant merge.
- Issues et PR gérées via **GitHub CLI** (`gh`), avec assignation et labels.
- Les PR indiquent les issues clôturées (`Closes #id`).

9. Plan de livraison (macro)

1. Stabilisation auth + pages publiques.
2. Boards + workspaces (CRUD, permissions).
3. Temps réel et activité.
4. Templates et automatisations.
5. Optimisations + monitoring.

10. Risques & mitigations

- **OAuth instable** : config stricte des redirect URI, tests multi-env.
- **Temps réel** : introduire progressivement via WebSocket + feature flags.
- **Complexité UI** : composants UI standardisés (shadcn/ui).

11. Livrables

- Frontend complet (Next.js).
- Backend GraphQL + DB.
- Infra Terraform GCP.
- Documentation technique + scripts.

Document évolutif, mis à jour au fil des features et livraisons.